

ENSEA

Beyond Engineering

REAL-TIME OPERATING SYSTEMS

Session 3 – Synchronisation

Laurent Fiack – laurent.fiack@ensea.fr

Bureau D212

Where we are

- Session 2: the APP is an **active object**
  - Its own task, its own private queue, others post events
  - Its handlers **run to completion**
- The BSP has its own task too: polls inputs every 5 ms
- But one handler still blocks for a long time: drawing the screen
  - `bsp_oled_show()` sends the whole framebuffer over I^2C, in polling: ≈ 25 ms
- Today:
  - Let a task **wait for an interrupt**: semaphores, notifications
  - **Protect shared data**: reentrancy, critical sections, mutexes
  - Apply both to the display: DMA + a dedicated display task

ENSEA
Beyond Engineering

Today

1. Waiting for an Interrupt
2. Sharing Data
3. Application: the Display

ENSEA
Beyond Engineering

Deferred processing

- An ISR must be **short**: it blocks every less urgent interrupt, and every task
- The long processing is **deferred** to a task. The ISR only says "something happened"

Naive version: a flag

```
volatile bool char_available = false;

void USART1_IRQHandler(void) {
    char_available = true;
    /* ... */
}

void task_usart1(void *unused) {
    for (;;) {
        if (char_available) {      /* polling */
            char_available = false;
            process_char(input_char);
        }
    }
}
```

What is wrong?

- The task never blocks: 100 % CPU, Idle never runs
- If it is high priority, lower tasks **starve**
- If it is low priority, the latency depends on the others

We need the task to be **Blocked** until the interrupt happens.

ENSEA
Beyond Engineering

The right way: a binary semaphore

```
SemaphoreHandle_t sem_usart1;

void USART1_IRQHandler(void) {
    BaseType_t woken = pdFALSE;
    xSemaphoreGiveFromISR(sem_usart1, &woken);
    /* ... */
    portYIELD_FROM_ISR(woken);
}

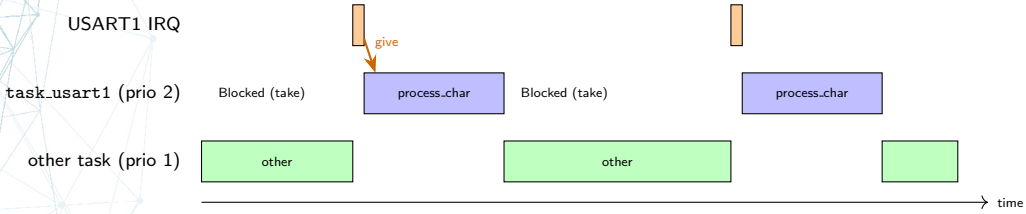
void task_usart1(void *unused) {
    for (;;) {
        xSemaphoreTake(sem_usart1, portMAX_DELAY);
        process_char(input_char);
    }
}

/* in main, before the scheduler */
sem_usart1 = xSemaphoreCreateBinary();
```

- The task is **Blocked** on `xSemaphoreTake`: no CPU used
- The ISR **gives** the semaphore: the task becomes Ready
- With `portYIELD_FROM_ISR`, it runs right after the ISR if it is the most urgent
- Same `FromISR` rules as for queues (session 2)

ENSEA
Beyond Engineering

Binary semaphore: the picture



- A binary semaphore holds 0 or 1 "token". **Empty** when created
- `Take` on an empty semaphore: Blocked. `Give` on a full one: nothing happens (fails)
- Two interrupts before the task runs = **one** wake-up: the events merge

Semaphore API

```c
#include "semphr.h"

SemaphoreHandle_t s = xSemaphoreCreateBinary();      /* empty, NULL if no heap */

BaseType_t xSemaphoreTake(s, timeout);   /* pdTRUE: taken; pdFALSE: timeout, NOT taken */
BaseType_t xSemaphoreGive(s);            /* pdTRUE: given; pdFALSE: already full      */

BaseType_t xSemaphoreGiveFromISR(s, &woken);         /* from an ISR, never blocks */
```

- Same timeouts as queues: 0, a duration, `portMAX_DELAY`
- After a timeout, the semaphore is **not** taken: check the return value

Under the hood

In FreeRTOS, a binary semaphore **is** a queue of length 1 whose items have size 0. Reusing the queue code saves Flash.

ENSEA
Beyond Engineering

Task notifications: the same, lighter

```
TaskHandle_t usart1_task;   /* the target */

void USART1_IRQHandler(void) {
    BaseType_t woken = pdFALSE;
    vTaskNotifyGiveFromISR(usart1_task, &woken);
    /* ... */
    portYIELD_FROM_ISR(woken);
}

void task_usart1(void *unused) {
    for (;;) {
        ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
        process_char(input_char);
    }
}
```

- Every task has a built-in notification value, in its TCB
- No object to create: faster, less RAM
- `pdTRUE`: clear the value on exit (behaves like a binary semaphore)
- Added later (v8.2) as an optimisation

ENSEA
Beyond Engineering

Notification or semaphore?

	Notification	**Semaphore**
Object to create	no	yes
Speed, RAM	better	good
The sender must know	**the task** (handle)	only the semaphore
Several receivers	no	yes
Channels per task	**one**[1]	as many as needed

- Notification: the default choice when *one* event source wakes *one* known task
- Semaphore: when the sender should not know the receiver, or when a task waits on several sources

[1] Indexed notifications (several per task) only exist since FreeRTOS 10.4. Our CubeMX ships 10.3.1.

ENSEA
Beyond Engineering

Counting semaphore

```
SemaphoreHandle_t s = xSemaphoreCreateCounting(uxMaxCount, uxInitialCount);
```

- Holds an integer between 0 and `uxMaxCount`
- Two typical uses:
  - **Counting requests**: each give is one event, none is merged (an occurrence, not a state)
  - **Managing resources**: initial count = number of free resources; take one, give it back
- With notifications: `ulTaskNotifyTake(pdFALSE, ...)` decrements instead of clearing – a counting semaphore

ENSEA
Beyond Engineering

Back to session 2

- `q->count++` in one task, `q->count--` in another
- Read – modify – write, preempted in the middle: an event is lost
- We used a FreeRTOS queue, which is already protected. Today: *how* do we protect our own data?

Vocabulary

- **Critical resource**: a variable, a structure, a peripheral shared by several tasks (or ISRs)
- **Critical section**: the piece of code that accesses it
- **Mutual exclusion**: only one task at a time in the critical section

ENSEA
Beyond Engineering

Reentrancy: the swap example

Not reentrant

```
int tmp;                        /* shared! */

void swap(int *x, int *y) {
    tmp = *x;
    *x = *y;
    /* interrupt: isr() runs here */
    *y = tmp;                   /* tmp was overwritten */
}

void isr(void) {
    int x = 1, y = 2;
    swap(&x, &y);
}
```

Reentrant

```
void swap(int *x, int *y) {
    int tmp;                    /* on the stack */
    tmp = *x;
    *x = *y;
    /* interrupt: isr() runs here */
    *y = tmp;                   /* still correct */
}

void isr(void) {
    int x = 1, y = 2;
    swap(&x, &y);
}
```

- Each call has its own `tmp` on its own stack: interrupting and calling again is harmless

ENSEA
Beyond Engineering

Reentrant vs. thread-safe

- **Reentrant**: can be interrupted, then called again (ISR, other task) before it finishes, without problem
  - It touches **no shared state**: only parameters and local variables
  - Example: `hsv_to_rgb`
- **Thread-safe**: can be called by several tasks at the same time without corruption
  - It touches shared state, but **protects** it (critical section, mutex)
  - Example: `xQueueSend`

Reentrant ⇒ thread-safe, not the other way round

- A function protected by a mutex is thread-safe, but cannot be called from an ISR (e.g. `malloc` with a lock)
- Our old `event_queue_push` was neither

ENSEA
Beyond Engineering

Protection 1: critical section

```
taskENTER_CRITICAL();
q->count++;                    /* nothing can interrupt us here... almost */
taskEXIT_CRITICAL();
```

- Masks the interrupts **and** therefore the context switches
- But **not all** interrupts: only those with a priority numerically ≥ 5
  - IRQs 0 to 4 keep running: guaranteed low latency for the most critical ones
  - They may not call FreeRTOS, so they cannot disturb the kernel
- Very cheap, but everybody waits: keep it **a few instructions long**
- From an ISR: `taskENTER_CRITICAL_FROM_ISR()`

ENSEA
Beyond Engineering

Protection 2: suspending the scheduler

```
vTaskSuspendAll();
/* ... critical section ... */
xTaskResumeAll();
```

- No context switch, but interrupts keep running
- A more urgent task that becomes Ready must **wait**: priorities are ignored
- Must not call a blocking API inside (there is no one to switch to)
- Does not protect against ISRs

To be avoided

Like the critical section, it penalises *every* task, even those that never touch the resource.

ENSEA
Beyond Engineering

Protection 3: a mutex

```
SemaphoreHandle_t m = xSemaphoreCreateMutex();      /* full when created */

xSemaphoreTake(m, portMAX_DELAY);
/* ... critical section ... */
xSemaphoreGive(m);
```

- Interrupts keep running, preemption keeps working
- A more urgent task still runs – it only blocks **if** it tries to take the same mutex
- Only the tasks that share the resource pay the price
- The critical section may be long, and may even block

The right tool for mutual exclusion between tasks

But not usable from an ISR: an ISR cannot block.

ENSEA
Beyond Engineering

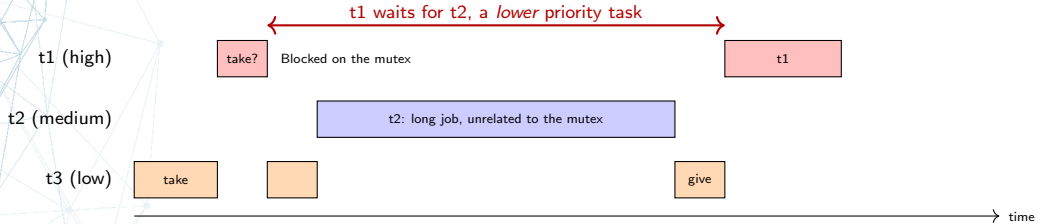
Binary semaphore vs. mutex

	Binary semaphore	**Mutex**
Purpose	**synchronisation**	**mutual exclusion**
Initial state	empty (0)	full (1)
Pattern	producer gives, consumer takes	take, use, give
Who gives?	anybody, including an ISR	**only the task that took it**
From an ISR	give: yes	no
Priority inheritance	no	**yes**

- Same API (`Take` / `Give`), very different meanings: do not mix them up

ENSEA
Beyond Engineering

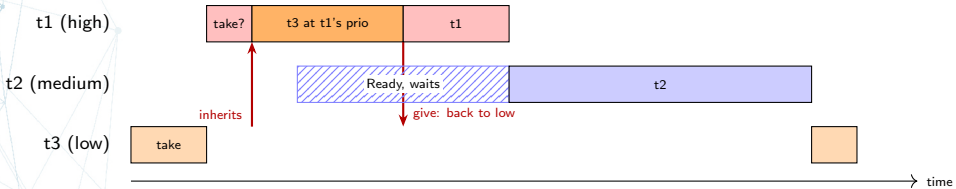
Priority inversion



- t3 holds the mutex; t1 waits for it – fine, for a short time
- But t2 preempts t3: t1 now waits for t2, whose priority is **lower** than its own
- Famous case: Mars Pathfinder (1997), repeated resets on Mars

ENSEA
Beyond Engineering

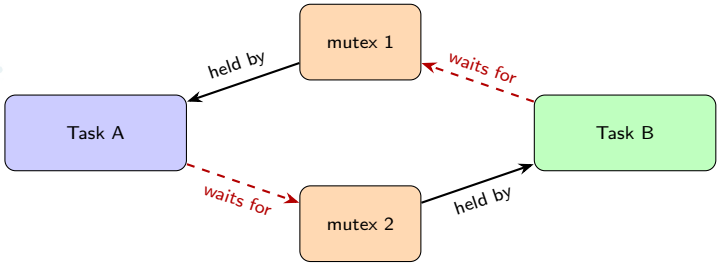
Priority inheritance



- While t1 waits for the mutex, t3 **inherits** t1's priority: t2 cannot preempt it
- t3 releases the mutex quickly, gets its own priority back, and t1 runs
- Built into FreeRTOS mutexes – not into binary semaphores

ENSEA
Beyond Engineering

Deadlock



- A holds mutex 1 and waits for mutex 2; B holds mutex 2 and waits for mutex 1: both wait forever
- Solution: every task takes the mutexes **in the same order**
- Better: avoid holding several mutexes at once – or use timeouts and check them

ENSEA
Beyond Engineering

The problem

- `ssd1306_UpdateScreen()`: for each of the 8 pages, 3 commands then 128 bytes of pixels
- Everything in polling: `HAL_I2C_Mem_Write(..., HAL_MAX_DELAY)`
- $\approx$ 25 ms of busy waiting, called from `bsp_oled_show()`...
- ... called by the APP, inside its event handlers

Questions

- Which task runs `ssd1306_UpdateScreen`? What does it break?
- How do we use the DMA without rewriting the driver as a state machine?

ENSEA
Beyond Engineering

Step 1: DMA + notification

With callbacks only

- Start the DMA for page 0, return
- In the completion callback: start page 1…
- The sequential loop becomes a **state machine** spread over callbacks

With an RTOS

- Start the DMA, then `ulTaskNotifyTake`: the task is **Blocked**
- The completion ISR notifies the task
- The loop over the pages stays **sequential**, unchanged
- The CPU is free during the transfer

- The 3 commands per page stay in polling: tolerated here to go fast (ideally, no polling at all)

ENSEA
Beyond Engineering

Step 2: who waits? A thought experiment

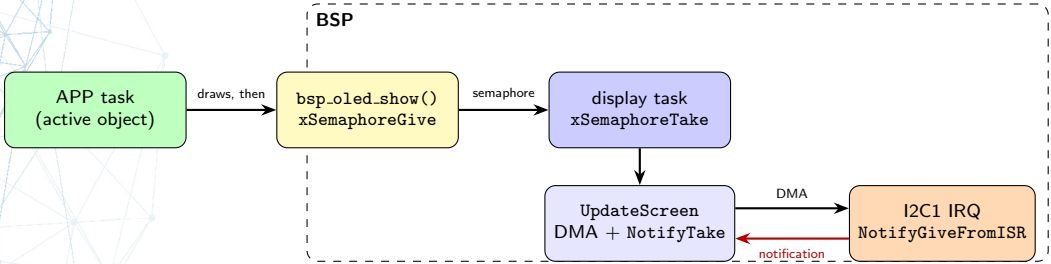
1. **The APP waits** for the notification
   - The CPU is free... but the active object is blocked $\approx 25\,\text{ms}$ per refresh
   - Violates **run-to-completion**
2. **A dedicated display task** waits; the APP only requests a refresh
   - But the framebuffer is **shared**: the APP draws while the display task sends it
3. **A mutex** on the framebuffer
   - The APP blocks while the frame is being sent: violates run-to-completion again
4. **Double buffer**: the APP draws in one buffer while the other is sent; swap at the end of the transfer
   - The right solution

Implemented today

Dedicated display task, **single** buffer: we tolerate glitches to stay simple – and they will show why the double buffer is needed.

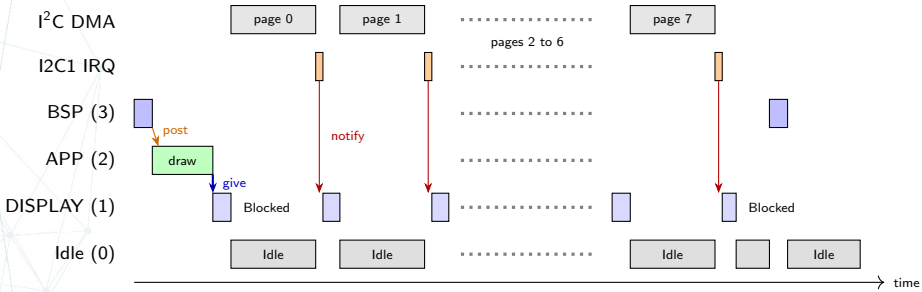
ENSEA
Beyond Engineering

Architecture



- The display task belongs to the **BSP**: the screen, the I^2C, the DMA and the IRQ are hardware
- `bsp.h` does not change: the APP still draws, then calls `bsp_oled_show()`

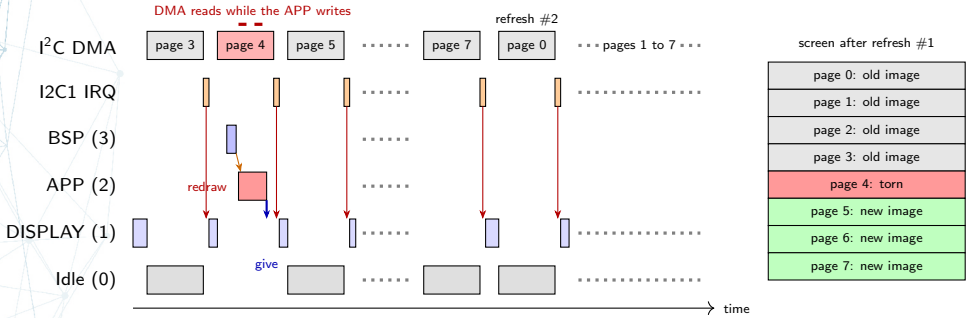
What happens, in time



- The APP only draws and gives: `bsp_oled_show()` returns immediately
- For each page, the display task only sends 3 commands and starts the DMA, then blocks
- During each transfer, the CPU is free: Idle – or any other task – runs

ENSEA
Beyond Engineering

Where the glitches come from



- The APP (prio 2) preempts and redraws **while** the DMA reads the same framebuffer: one page is **torn**
- The `give` is merged: refresh #2 fixes the screen – the glitch lasts one frame

The display task

```c
/* Called by the APP: no transfer anymore, returns immediately (run-to-completion) */
void bsp_oled_show(void) {
    xSemaphoreGive(bsp_ctx.display_request);
}

static void bsp_display_task(void *unused) {
    (void)unused;
    for (;;) {
        xSemaphoreTake(bsp_ctx.display_request, portMAX_DELAY);   /* Blocked: waits for the APP */
        if (bsp_ctx.display_present) {
            ssd1306_UpdateScreen();                               /* sequential code, unchanged */
        }
    }
}
```

- `bsp_init_display()` creates the semaphore and the task, and stores the task handle in `bsp_ctx`

ENSEA
Beyond Engineering

The transfer and its interrupt

```
void ssd1306_WriteData(uint8_t *buffer, size_t size) {    /* replaces the driver's version */
    configASSERT(xTaskGetSchedulerState() != taskSCHEDULER_NOT_STARTED);

    if (HAL_I2C_Mem_Write_DMA(&hi2c1, SSD1306_I2C_ADDR, 0x40, 1, buffer, size) != HAL_OK) {
        return;
    }
    ulTaskNotifyTake(pdTRUE, portMAX_DELAY);              /* Blocked during the DMA: CPU free */
}

void HAL_I2C_MemTxCpltCallback(I2C_HandleTypeDef *hi2c) { /* in bsp.c, called from an ISR */
    if (hi2c->Instance != I2C1) return;
    BaseType_t woken = pdFALSE;
    vTaskNotifyGiveFromISR(bsp_ctx.display_task, &woken); /* fixed handle */
    portYIELD_FROM_ISR(woken);
}
```

- The callback is called from `I2C1_EV_IRQHandler` (end of transfer, STOP condition), not from the DMA ISR: **I2C1_EV** needs a priority ≥ 5
- `HAL_I2C_ErrorCallback` wakes the task too – otherwise an I²C error blocks it forever

ENSEA
Beyond Engineering

Why a semaphore *and* a notification?

- The display task waits for **two** different things:
  - a refresh request, from the APP
  - the end of the DMA, from the ISR
- A task has only **one** notification (FreeRTOS 10.3.1)
  - If the APP notified too, a request arriving during a transfer would be taken for the end of the DMA
- Each mechanism where it fits best:
  - ISR → task: **notification** – the ISR knows exactly whom to wake (fixed handle)
  - APP → task: **semaphore** – the APP does not need to know that a display task exists

ENSEA
Beyond Engineering

Why not an active object for the display?

- We could give the display task a queue of events, like the APP. But:
  1. No need to **store** refresh requests: a refresh is a **state**, only the last one matters (session 2: occurrence vs. state)
  2. There is only **one** kind of event
- The binary semaphore merges the requests: several `show` during a transfer = one refresh

A wink

In FreeRTOS, a binary semaphore *is* a queue of length 1 with no data: the display task is a degenerate active object.

ENSEA
Beyond Engineering

Replacing the driver's transport

- We replace `ssd1306_WriteData`, not `ssd1306_UpdateScreen`
  - The driver keeps the **protocol** (pages, commands); we only change the **transport**
- In the driver: `ssd1306_WriteData` is declared `__weak` – one word added
- In `bsp.c`: our own `ssd1306_WriteData`, with DMA + notification
  - The linker picks ours: **link-time polymorphism**
  - The driver knows neither the DMA nor FreeRTOS
- The `ssd1306_UpdateScreen()` inside `ssd1306_Init()` is commented out
  - Init runs before the scheduler: no task to block. The first refresh comes from the APP

Discussion

In a well-written driver, the transport would be **injected** by function pointer at init. Not done here: too heavy for the lab.

ENSEA
Beyond Engineering

Priorities and init

```
bsp_init(3);            /* inputs: short, blocks often: the most urgent */
bsp_init_display(1);    /* display: long, not urgent -- before app_init */
app_init(2);            /* active object                                */
vTaskStartScheduler();
```

- Each module creates its own task; `main` chooses the priorities
- The display gets the **lowest** priority: it runs when inputs and APP have nothing to do
- `bsp_init_display` before `app_init`: the APP draws its first screen in its init

ENSEA
Beyond Engineering

Pitfall: HAL_Delay stuck before the scheduler

- Symptom: `bsp_init(3)` creates its task, then `ssd1306_Init()` never leaves its `HAL_Delay(100)`
- Cause:
  - Every FreeRTOS API call (`xTaskCreate`, `xSemaphoreCreate...`) enters a critical section: IRQs ≥ 5 masked
  - The nesting counter starts at `0xaaaaaaaa`, reset to 0 only by `vTaskStartScheduler()`
  - So the IRQs stay masked until the scheduler starts – on purpose: no context switch before the scheduler exists
  - The HAL time base (TIM6) was at priority 15: `uwTick` frozen, `HAL_Delay` never ends
- Fix: TIM6 at priority **4** (CubeMX, NVIC, time base)
  - The HAL time base never calls FreeRTOS: no reason to be ≥ 5

Rule

Between the first FreeRTOS call and `vTaskStartScheduler()`, the interrupts allowed to call FreeRTOS are masked. Same reason why no pixel can be sent by DMA before the scheduler.

ENSEA
Beyond Engineering

Result

- The APP stays responsive: `bsp_oled_show()` returns immediately
- The CPU is free during the transfers
- But **glitches** appear: the APP draws in the framebuffer while the DMA sends it

Next step

Double buffer: draw in one buffer, send the other, swap at the end of the transfer.

ENSEA
Beyond Engineering

Summary

- Defer long processing from ISRs to tasks: the task **blocks**, the ISR wakes it
- Binary semaphore: synchronisation, merges events. Counting semaphore: counts them
- Notification: lighter, but one channel per task and the sender must know the task
- Reentrant (no shared state) vs. thread-safe (protected shared state)
- Critical section, scheduler suspension, **mutex**: the mutex only penalises the tasks sharing the resource
- Mutex $\neq$ binary semaphore: ownership, priority inheritance; beware of deadlocks
- Display: DMA + notification keeps the code sequential; a dedicated task keeps the APP running to completion

ENSEA
Beyond Engineering